

UTFPR - UNIVERSIDADE TECNOLÓGICA  
FEDERAL DO PARANÁ

Bacharelado em Engenharia de Software - 6º Período

**DISCIPLINA: Oficina de Integração 1 - ES46F-ES61**

**Professor: Eduardo Cotrin Teixeira**

---

## Documento de Projeto de Software

---

Cursinho Prisma

Gustavo Pukanski Schatzmann  
Igor Busquim de Moraes  
Luiz Fernando Moreira Domenico  
Orlando Cardoso Neto  
Rafael Spitzer Cardoso da Silva

Cornélio Procópio  
2026

# Índice

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                 | <b>3</b>  |
| 1.1      | Contexto . . . . .                                | 3         |
| 1.2      | Justificativa . . . . .                           | 3         |
| 1.3      | Proposta . . . . .                                | 3         |
| 1.4      | Organização do Documento . . . . .                | 3         |
| <b>2</b> | <b>Descrição Geral do Sistema</b>                 | <b>5</b>  |
| 2.1      | Objetivos (Gerais e Específicos) . . . . .        | 5         |
| 2.2      | Objetivo Geral . . . . .                          | 5         |
| 2.3      | Objetivos Específicos . . . . .                   | 5         |
| 2.4      | Limites e Restrições . . . . .                    | 5         |
| 2.5      | Limites . . . . .                                 | 5         |
| 2.6      | Restrições . . . . .                              | 6         |
| 2.7      | Descrição dos Usuários do Sistema . . . . .       | 6         |
| <b>3</b> | <b>Desenvolvimento do Projeto</b>                 | <b>7</b>  |
| 3.1      | Tecnologias e ferramentas . . . . .               | 7         |
| 3.1.1    | Serviço de Inteligência Artificial . . . . .      | 7         |
| 3.1.2    | Aplicação Web e API . . . . .                     | 7         |
| 3.1.3    | Banco de Dados e Armazenamento . . . . .          | 7         |
| 3.1.4    | Gerenciamento, Qualidade e Documentação . . . . . | 7         |
| 3.2      | Metodologia de desenvolvimento . . . . .          | 8         |
| 3.3      | Cronograma previsto . . . . .                     | 9         |
| <b>4</b> | <b>Requisitos do Sistema</b>                      | <b>10</b> |
| 4.1      | Requisitos Funcionais . . . . .                   | 10        |
| 4.2      | Requisitos Não-funcionais . . . . .               | 13        |
| 4.3      | Diagramas de Casos de Uso . . . . .               | 14        |
| 4.4      | Protótipos de Telas . . . . .                     | 15        |
| 4.4.1    | Tela de Login . . . . .                           | 15        |
| 4.4.2    | Dashboard do Professor . . . . .                  | 15        |
| 4.4.3    | Tela de Cadastro de Questões . . . . .            | 16        |
| 4.4.4    | Tela de Criar Listas . . . . .                    | 16        |
| 4.4.5    | Tela do Banco de Questões . . . . .               | 17        |
| 4.4.6    | Tela de Gerenciamento de Frequência . . . . .     | 18        |
| 4.4.7    | Dashboard do Aluno . . . . .                      | 18        |
| 4.4.8    | Tela de Resolução de Exercícios . . . . .         | 19        |
| 4.4.9    | Painel Administrativo . . . . .                   | 19        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| <b>5</b> | <b>Análise do Sistema</b>          | <b>21</b> |
| 5.1      | Modelo do Banco de Dados . . . . . | 21        |
| 5.2      | Diagrama de Classes . . . . .      | 21        |
| 5.3      | Diagrama de Atividades . . . . .   | 21        |
| <b>6</b> | <b>Implementação</b>               | <b>22</b> |
| 6.1      | Descrição do código . . . . .      | 22        |
| 6.2      | Implantação . . . . .              | 22        |
| 6.3      | Telas principais . . . . .         | 22        |
| <b>7</b> | <b>Considerações Finais</b>        | <b>23</b> |
| <b>8</b> | <b>Bibliografia</b>                | <b>24</b> |

# **1 Introdução**

## **1.1 Contexto**

O projeto irá ser implementado no ambiente educacional do projeto de extensão do Cursinho Prisma, focado em pré-vestibular. Atualmente, a rotina de preparação de materiais, montagem de listas e o acompanhamento de rendimento dos alunos é feito de forma manual e descentralizada. Paralelamente, os alunos e a coordenação não possuem métricas automatizadas sobre o desempenho em disciplinas e matérias específicas. O sistema será implementado neste cenário para atuar como uma plataforma web que reduz o atrito na criação de materiais e fornece inteligência de dados educacionais

## **1.2 Justificativa**

O problema central é a ineficiência e o alto custo de tempo na extração, categorização e distribuição de questões de vestibulares, somado a falta de visibilidade sobre o desempenho real dos alunos. Atualmente, os professores são sobrecarregados pela triagem manual do material, e os alunos, não conseguem direcionar seus estudos com base em dados reais de sua proficiência. Essa falta de otimização dos materiais e a falta de autoanálise acaba afetando o nivelamento acadêmico e pode acabar gerando uma evasão na parte do aluno. Ao simplificar a criação de questões e automatizar a etapa de categorização, o sistema reduz o esforço do professor. Quando o sistema estiver operante, ele irá ter um banco de questões auto-organizado, que permite a geração rápida de listas e dashboards de desempenho

## **1.3 Proposta**

Uma aplicação web com um módulo de cadastro de questões na parte do professor, contendo uma interface otimizada onde o professor ou monitor insere o texto e imagens da questão. O sistema realiza uma chamada a um modelo de IA que analisa o texto e sugere automaticamente a classificação correta (Disciplina Macro e Micro) e o nível de dificuldade. Além deste módulo, teremos os módulos do professor, com gestão de turmas, geração dinâmica de listas de exercícios e painéis analíticos, e também do aluno, com o banco de questões, listas dos professores e a visão de autodesempenho

## **1.4 Organização do Documento**

O capítulo 1 apresenta o contexto do Cursinho Prisma, a justificativa do problema a ser resolvido e a proposta de solução baseada em IA e gestão de

dados. O capítulo 2 detalha os objetivos gerais específicos do sistema, expõe as limitações técnicas impostas pelo processamento da IA, e define os papéis dos usuários envolvidos na operação do sistema. O capítulo 3 detalha as tecnológicas, a metodologia ágil paralela e o cronograma. O capítulo 4 especifica os requisitos funcionais e não-funcionais que guiarão o desenvolvimento.

## 2 Descrição Geral do Sistema

### 2.1 Objetivos (Gerais e Específicos)

#### 2.1.1 Objetivo Geral

Desenvolver e implantar um sistema web para gestão educacional focado em avaliação de desempenho e criação de listas de exercícios, sustentado por um banco de questões alimentado colaborativamente através de uma interface assistida por IA para automação da categorização e da dificuldade.

#### 2.1.2 Objetivos Específicos

Metas e ações que, somadas, permitem alcançar o objetivo geral:

- Desenvolver uma interface web otimizada para inserção rápida de questões (textos e imagens manuais);
- Integrar um modelo de IA local (como o Ollama) à API para processar textos de questões inseridas e retornar sugestões estruturadas de categorização e dificuldade;
- Desenvolver o módulo do professor, permitindo o agrupamento de questões em listas e a gestão de turmas;
- Desenvolver o módulo do aluno para resolução de exercícios;
- Implementar dashboards de proficiência baseados no histórico de resoluções.

### 2.2 Limites e Restrições

#### 2.3 Limites

- **Limitação de extração:** O sistema não realiza, atualmente, a extração autônoma, leitura de PDFs e recortes automáticos ou OCR de provas. Toda questão deve ser inserida na interface pelo usuário, seja por digitação ou copiando e colando, com upload manual de imagens pelo usuário;
- **Limitação da IA:** A IA atuará estritamente como um assistente de preenchimento. O usuário terá a palavra final para aceitar, alterar ou recusar a categoria e a dificuldade proposta pelo modelo antes de salvar no banco;

- **Interação síncrona:** Está fora do escopo a criação de fóruns, chats em tempo real, ou transmissão de aulas;
- **Integração externa:** O sistema não fará integração com sistemas acadêmicos governamentais ou plataformas externas de vestibulares.

## 2.4 Restrições

- Pela falta de recursos, a extração automática de questões diretamente de PDFs de provas não será feita atualmente, podendo ser uma funcionalidade futura;
- A acurácia de categorização não será de 100%, sendo necessária a intervenção manual humana antes da persistência no banco de dados;
- A API que conecta o sistema ao Ollama exige que o servidor de hospedagem tenha capacidade mínima de processamento para inferência do modelo, ou que a API de IA seja isolada em um serviço em nuvem específico;
- Pela falta de recursos, o sistema irá se restringir ao uso de ferramentas gratuitas ou com tiers gratuitos.

## 2.5 Descrição dos Usuários do Sistema

- **Aluno:** Acessa a plataforma web para resolver as listas designadas pelos professores, treina no banco livre e acompanha o seu desempenho pessoal nas disciplinas, em nível macro e micro;
- **Professor/Monitor:** Acessa a plataforma web para analisar o desempenho das turmas por meio do dashboard, identifica discrepâncias no desempenho, cadastra novas questões no banco, gera listas de exercícios personalizadas e direcionadas consultando o banco de questões;
- **Administrador (Coordenação):** Gerencia os cadastros de professores e alunos, define a árvore de macro e micro disciplinas no banco e administra o sistema.

### 3 Desenvolvimento do Projeto

#### 3.1 Tecnologias e ferramentas

##### 3.1.1 Serviço de Inteligência Artificial

- **Linguagem:** Python com FastAPI, atuando como uma API interna;
- **Categorização semântica e de dificuldade:** Ollama;
- **Hospedagem:** Ngrok.

##### 3.1.2 Aplicação Web e API

- **Linguagem backend:** ASP .NET;
- **Linguagem frontend:** Angular;
- **Hospedagem front-end:** Vercel;
- **Hospedagem back-end:** Render;
- **Containerização:** Docker.

##### 3.1.3 Banco de Dados e Armazenamento

- **SGBD relacional:** PostgreSQL;
- **Armazenamento de arquivos:** Supabase;
- **Hospedagem:** Supabase.

##### 3.1.4 Gerenciamento, Qualidade e Documentação

- **Versionamento:** Git com repositório no GitHub;
- **Gerenciamento:** Jira;
- **Testes de API:** Insomnia;
- **Documentação de API:** Swagger.



### 3.2 Metodologia de desenvolvimento

O projeto utilizará uma adaptação do Scrum para o contexto acadêmico, com sprints semanais. A escolha dessa abordagem se deve à incerteza técnica envolvendo as diferentes tecnologias adotadas. Uma metodologia mais rígida poderia comprometer o andamento do projeto caso ocorra alguma falha em um módulo específico.

Para otimizar o trabalho, o desenvolvimento não será estritamente sequencial, sendo dividido em quatro frentes paralelas que se integram ao longo do tempo:

1. **Frente Front-End (2 pessoas):** Responsável pela prototipação das telas, gerenciamento do estado da aplicação e consumo dos endpoints da API ASP.NET;
2. **Frente Back-End:** Responsável pela modelagem do banco de dados, criação dos CRUDs, implementação de segurança e documentação via Swagger;
3. **Frente Serviço de IA:** Responsável pela criação da API interna que se comunica com o Ollama, ajuste de prompts para garantir respostas estruturadas e consistentes, além da otimização do tempo de resposta da inferência;
4. **Frente de Integração e Infraestrutura:** Responsável pela configuração das ferramentas, execução de testes e integração entre as diferentes frentes de desenvolvimento.

### 3.3 Cronograma previsto

| Semana         | Frente Front-End                  | Frente Back-End                     | Frente IA                        | Frente Integração e Infra            |
|----------------|-----------------------------------|-------------------------------------|----------------------------------|--------------------------------------|
| 1              | Entender o escopo e o sistema     | Entender o escopo e o sistema       | Entender o escopo e o sistema    | Entender o escopo e o sistema        |
| 2              | Criar protótipos das telas        | Modelar banco de dados              | Definir requisitos da IA         | Configurar repositórios              |
| 3 (Entrega 1)  | Telas base e login                | CRUD base e banco rodando           | Ajustar retorno da IA em JSON    | Conectar Front e Back                |
| 4              | Tela de cadastro de questões      | Endpoint para receber questões      | Tratar erros de texto na IA      | Criar Dockerfile e upload de imagens |
| 5              | Tela de revisão da IA             | Conectar C# com Python              | Testar IA com questões reais     | Conectar Back com IA                 |
| 6              | Telas de filtro e listas          | Lógica de criação de listas         | Ajustar erros e alucinações      | Validar fluxo do Front até a IA      |
| 7              | Tela de visualização de turmas    | Consultas no banco PostgreSQL       | Otimizar tempo de resposta da IA | Resolver problemas de rede e CORS    |
| 8 (Entrega 2)  | Corrigir erros visuais            | Estabilizar a API e os dados        | Congelar a versão do prompt      | Testar cadastro com IA               |
| 9              | Telas iniciais do aluno           | Rotas de listas pendentes           | Avaliar erros do cadastro        | Preparar contas para o deploy        |
| 10             | Tela de resolução de exercícios   | Endpoint para salvar respostas      | Ajustes finais no texto da IA    | Subir o postgres na nuvem            |
| 11             | Tela do banco livre de questões   | Filtros de busca e rotas de chamada | Documentar como rodar a IA       | Subir a API ASP no Render            |
| 12             | Gráficos e percentual de presença | Consultas de acertos e frequência   | Ajustar parâmetros do Ollama     | Testar sistema na nuvem com IA local |
| 13             | Dashboard e tela de chamada       | Criar índices de banco de dados     | Corrigir bugs residuais da IA    | Fazer testes de carga                |
| 14 (Entrega 3) | Polimento final para celular      | Limpeza de código e segurança       | Fechar escopo da IA              | Entregar o sistema                   |

Tabela 1: Cronograma de desenvolvimento por frentes de trabalho

## 4 Requisitos do Sistema

### 4.1 Requisitos Funcionais

| ID   | Funcionalidade                                                                                                                    | Prioridade |
|------|-----------------------------------------------------------------------------------------------------------------------------------|------------|
| RF01 | O sistema deve prover uma interface para inserção manual do enunciado, alternativas e gabarito, além de upload de imagem de apoio | Essencial  |
| RF02 | Ao preencher o enunciado, o sistema deve acionar a IA para sugerir a disciplina, matéria e dificuldade (Fácil/Médio/Difícil)      | Essencial  |
| RF03 | O professor deve poder aceitar, editar ou recusar as sugestões da IA antes de salvar a questão                                    | Essencial  |
| RF04 | Professores e administradores devem poder listar, editar e desativar questões já salvas no banco                                  | Essencial  |
| RF05 | O sistema deve gerenciar autenticação e autorização para três perfis: Administrador, Professor e Aluno                            | Essencial  |
| RF06 | Professores devem poder criar turmas e adicionar alunos a elas                                                                    | Importante |
| RF07 | Professores devem poder buscar questões no banco através de filtros e agrupá-las em listas atribuídas a turmas específicas        | Importante |
| RF08 | Alunos devem poder abrir listas ou o banco livre, selecionar a alternativa e receber a correção automática no sistema             | Essencial  |
| RF09 | O sistema deve exibir gráficos para o aluno detalhando sua porcentagem de acertos/erros agrupados por macro e micro disciplinas   | Essencial  |
| RF10 | O sistema deve exibir relatórios de proficiência mostrando as maiores lacunas de aprendizado da turma e alunos individuais        | Essencial  |
| RF11 | A coordenação deve poder gerenciar contas de usuários e a estrutura de disciplinas do banco                                       | Essencial  |
| RF12 | O sistema deve fornecer uma interface rápida para o professor registrar a presença ou falta dos alunos                            | Desejável  |
| RF13 | No dashboard do aluno deve haver um indicador exibindo seu percentual de presença                                                 | Desejável  |

Tabela 2: Requisitos funcionais do sistema

- **RF01 - Interface de Cadastro Manual e Upload:** O sistema deve fornecer um formulário interativo no front-end para que professores ou monitores insiram o texto bruto da questão, as opções de múltipla escolha e a indicação do gabarito correto. Além disso, deve haver um componente de upload que permita anexar uma imagem à questão, enviando o arquivo diretamente ao Supabase e salvando a URL no banco de dados.
- **RF02 - Classificação Assistida por IA:** No momento em que o usuário preenche o enunciado da questão no formulário, o front-end deve disparar uma requisição assíncrona. A API ASP .NET repassará o texto para a API Python, que utilizará o modelo Ollama para analisar o contexto. O sistema deve retornar um JSON com a sugestão da disciplina macro, matéria micro e nível de dificuldade, baseando-se estritamente nas categorias existentes no banco de dados.
- **RF03 - Revisão Humana:** A interface não deve salvar a questão automaticamente após o retorno da IA. As sugestões geradas pelo RF02 devem preencher os campos de seleção (dropdown) da tela, servindo apenas como um facilitador. O usuário terá a obrigatoriedade de revisar, alterar ou confirmar essas categorias manualmente antes de submeter o formulário para persistência no banco de dados.
- **RF04 - Gestão de Questões (CRUD Base):** O sistema deve permitir operações completas de manipulação do acervo. Usuários com perfil de Professor ou Administrador poderão visualizar todas as questões em formato de lista (com filtros), editar o conteúdo de questões e desativar ou excluir questões desatualizadas do banco.
- **RF05 - Autenticação e Autorização:** O sistema deve possuir uma tela de login centralizada. Utilizando tokens JWT, o back-end validará as credenciais e as permissões. A interface web deve se adaptar ao perfil logado: alunos não podem acessar telas de cadastro de questões, e professores não podem acessar o painel de criação de usuários da administração.
- **RF06 - Gestão de Turmas:** O perfil professor deve ter acesso a um painel para criar turmas (representando grupos lógicos de alunos). O professor poderá buscar alunos cadastrados no sistema e vinculá-los a essas turmas, facilitando a distribuição de atividades.
- **RF07 - Criação de Listas de Exercícios:** O professor deve acessar uma interface de busca no banco de questões consolidado. Através

de filtros (por disciplina, matéria e dificuldade), ele poderá selecionar múltiplas questões e agrupá-las em uma lista. Esta lista será então atribuída a uma ou mais turmas cadastradas (RF06).

- **RF08 - Resolução de Exercícios (Aluno):** O aluno logado deve acessar um painel com suas listas pendentes. Ao abrir uma lista ou buscar questões no banco livre, ele verá a questão formatada, selecionará uma alternativa e enviará a resposta. O sistema deve registrar a tentativa no banco de dados e retornar o gabarito, indicando acerto ou erro.
- **RF09 - Dashboard de Autodesempenho (Aluno):** O sistema deve processar o histórico de resoluções do aluno logado e renderizar gráficos visuais. O painel deve destacar a taxa global de proficiência e detalhar os dados por macro e micro disciplinas, orientando o estudo individual.
- **RF10 - Dashboard de Desempenho e Defasagem (Professor):** O sistema deve consolidar as métricas de todos os alunos de uma turma e gerar relatórios visuais para o professor. O objetivo é identificar tendências de erro coletivas e permitir intervenções pedagógicas direcionadas, além de possibilitar a análise individual de alunos com risco de evasão.
- **RF11 - Gestão Administrativa de Perfis e Estrutura:** A coordenação terá um painel exclusivo para realizar o CRUD de usuários e gerenciar a árvore de disciplinas, inserindo novas matérias no banco que orientarão tanto os filtros quanto as restrições da IA.
- **RF12 - Gerenciamento de Frequência (Chamada):** O sistema deve fornecer uma interface rápida para o professor selecionar uma turma, uma data específica e registrar a presença ou falta dos alunos, persistindo o histórico no banco de dados.
- **RF13 - Consulta de Frequência (Aluno):** No painel do aluno, deve haver um indicador visual exibindo seu percentual de presença acumulado, calculado a partir dos registros inseridos pelos professores.

## 4.2 Requisitos Não-funcionais

| ID    | Requisito                                                                                                                                                      | Categoria      |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| RNF01 | O tempo de espera pela sugestão da IA não deve travar a tela, exibindo indicação visual de carregamento                                                        | Usabilidade    |
| RNF02 | O front-end deve ser responsivo e seguir princípios que garantam o uso em dispositivos móveis, sem perda de funcionalidades em telas de smartphones            | Usabilidade    |
| RNF03 | O banco de dados deve impor restrições de chave estrangeira rígidas entre as micro e macro disciplinas para impedir categorização em categorias inexistentes   | Confiabilidade |
| RNF04 | O sistema não deve expor a comunicação direta com a IA; o front-end deve realizar requisições apenas à API, que atuará como intermediária segura para o Ollama | Segurança      |
| RNF05 | As senhas devem ser protegidas no banco de dados utilizando algoritmos de hashing                                                                              | Segurança      |
| RNF06 | O tempo de resposta das consultas da API para renderização do dashboard não deve ultrapassar 3 segundos                                                        | Desempenho     |

Tabela 3: Requisitos não funcionais do sistema

### 4.3 Diagramas de Casos de Uso

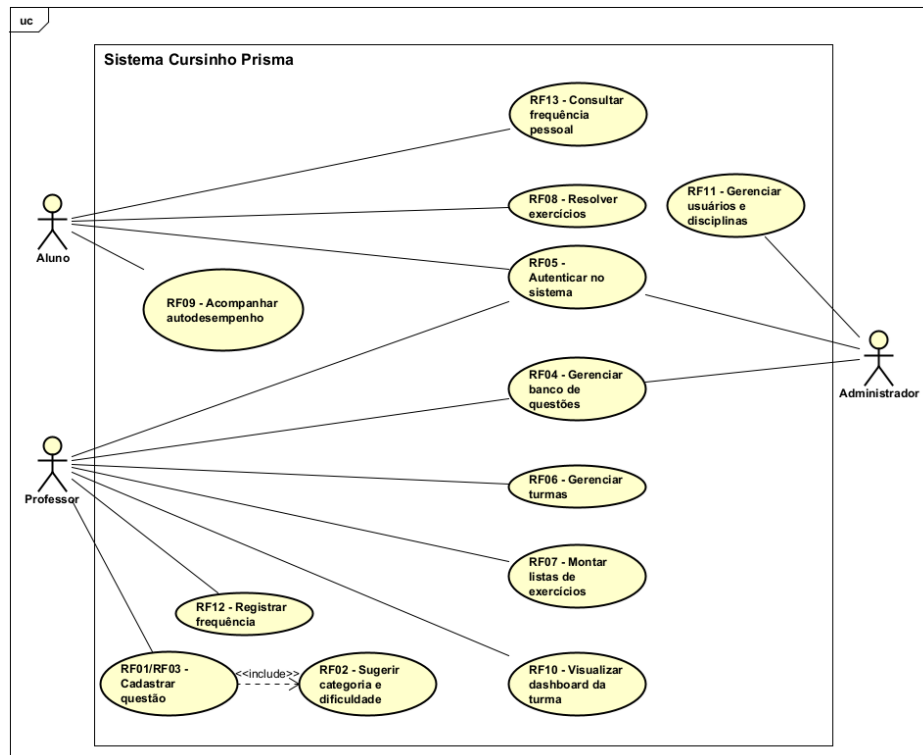


Figura 1: Diagrama de casos de uso representando as interações dos alunos, professores e administradores com o sistema cursinho prisma

## 4.4 Protótipos de Telas

### 4.4.1 Tela de Login

**Objetivo:** Autenticar os usuários e direcioná-los para o ambiente correto de acordo com o seu perfil.

**Dinâmica de Navegação:** É a tela inicial do sistema. A partir dela, dependendo da aba selecionada e da validação das credenciais, o usuário é redirecionado para o seu respectivo dashboard. Também possui link para recuperação de senha.

**Requisitos Funcionais Atendidos: RF05 - Autenticação e Autorização.**

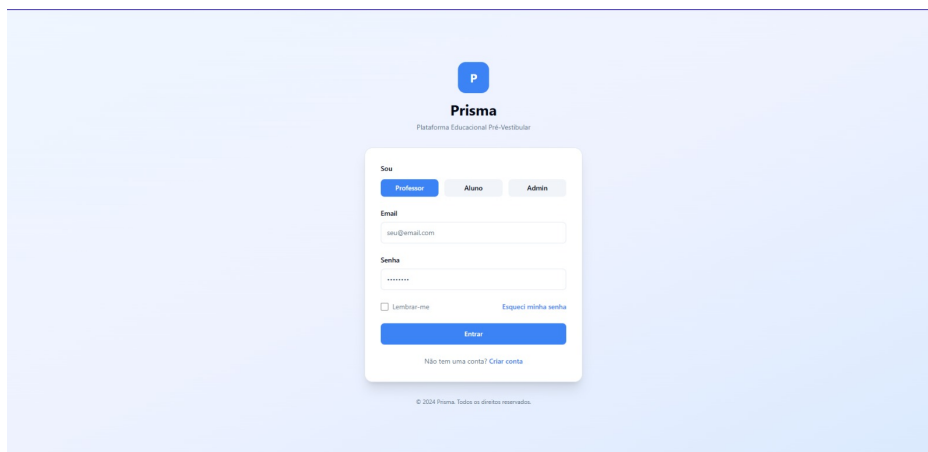


Figura 2: Tela de login

### 4.4.2 Dashboard do Professor

**Objetivo:** Fornecer um painel gerencial resumido para o educador, apresentando métricas-chave como total de alunos, questões cadastradas, desempenho médio das turmas e tendências de notas, além de alertas.

**Dinâmica de Navegação:** Tela principal após login do professor. O menu lateral permite acesso a funcionalidades como cadastro de questões, banco de questões e criação de listas. Possui botão de atalho “+ Nova Questão”.

**Requisitos Funcionais Atendidos: RF10 - Dashboard de Desempenho e Defasagem (Professor).**



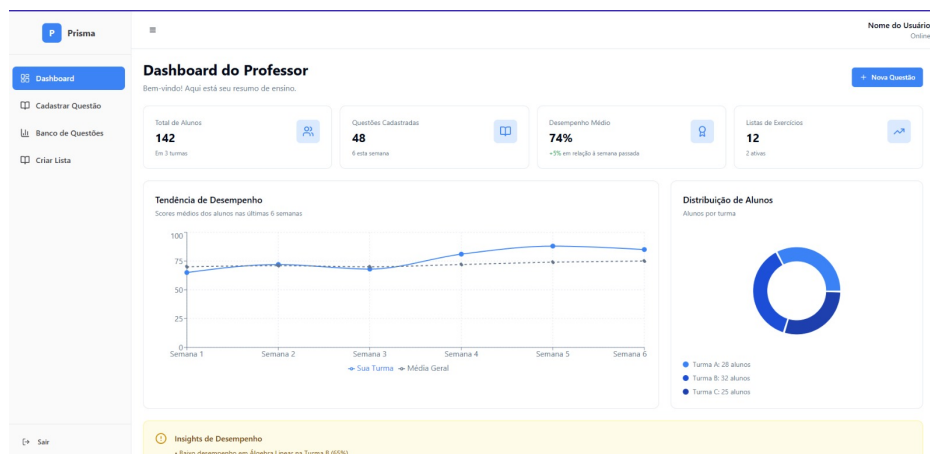


Figura 3: Dashboard do professor

#### 4.4.3 Tela de Cadastro de Questões

**Objetivo:** Permitir a inserção de novas questões no banco de dados, incluindo enunciado, alternativas, upload de imagem e assistência de IA.

**Dinâmica de Navegação:** Acessível pelo menu ou botão rápido. Após submissão, salva a questão e redireciona ou limpa o formulário.

**Requisitos Funcionais Atendidos:** RF01, RF02, RF03.

Figura 4: Tela de cadastro de questões

#### 4.4.4 Tela de Criar Listas

**Objetivo:** Agrupar questões em listas direcionadas a turmas.

**Dinâmica de Navegação:** Permite definir nome da lista, turma e adicionar questões via banco. Após gerar, redireciona para gestão.

**Requisitos Funcionais Atendidos: RF06, RF07.**

Figura 5: Tela de criação de listas

#### 4.4.5 Tela do Banco de Questões

**Objetivo:** Servir como repositório pesquisável de questões.

**Dinâmica de Navegação:** Permite busca, filtragem e edição de questões.

**Requisitos Funcionais Atendidos: RF04.**

Figura 6: Tela do banco de questões

#### 4.4.6 Tela de Gerenciamento de Frequência

**Objetivo:** Registrar presença e ausência dos alunos.

**Dinâmica de Navegação:** Seleção de turma e data, com marcação individual ou em massa.

**Requisitos Funcionais Atendidos: RF12.**

The screenshot displays the 'Gerenciamento de Frequência' (Frequency Management) interface. On the left is a sidebar with a 'Prisma' logo and navigation links: Dashboard, Cadastrar Questão, Banco de Questões, Criar Lista, and Frequência (highlighted). The main content area has a title 'Gerenciamento de Frequência' and a subtitle 'Registre a presença dos seus alunos'. It includes two tabs: 'Registrar Presença' (active) and 'Histórico'. Below the tabs are dropdowns for 'Selecione a Turma' (Turma A - Matemática (28 alunos)) and 'Selecione a Data' (06/04/2026). A summary section shows three colored boxes: 'Presenças' (0, green), 'Ausências' (0, red), and 'Não Marcados' (8, yellow). Below this is a green button 'Marcar Todos Presentes' and a 'Limpar' link. The 'Lista de Alunos' section lists five students: Ana Silva, Bruno Santos, Carlos Oliveira, Diana Costa, and Eduardo Rocha. Each student has 'Presente' and 'Ausente' buttons with checkmarks.

Figura 7: Tela de gerenciamento de frequência

#### 4.4.7 Dashboard do Aluno

**Objetivo:** Exibir desempenho, progresso e atividades pendentes.

**Dinâmica de Navegação:** Tela inicial do aluno com acesso às listas.

**Requisitos Funcionais Atendidos: RF09, RF13.**



Figura 8: Dashboard do aluno

#### 4.4.8 Tela de Resolução de Exercícios

**Objetivo:** Ambiente focado na resolução de questões.

**Dinâmica de Navegação:** Navegação por questões e submissão ao final.

**Requisitos Funcionais Atendidos:** RF08.

**Resolver Exercícios**  
Revisão - Equações Quadráticas

**Respostas:** 0/3

**Seu Progresso:** 0/3 respondidas

**Questão 1 de 3**

Resolver a equação quadrática:  $x^2 - 5x + 6 = 0$

Selecione uma alternativa:

☐ A)  $x = 2$  ou  $x = 3$

☐ B)  $x = 1$  ou  $x = 4$

☐ C)  $x = 2$  ou  $x = 4$

☐ D)  $x = 3$  ou  $x = 6$

☐ E)  $x = 1$  ou  $x = 2$

**Navegador:** 1 (selecionado), 2, 3

☐ Não respondida

☐ Respondida

☒ Resolvida

Botões: Anterior, Próxima

Figura 9: Tela de resolução de exercícios

#### 4.4.9 Painel Administrativo

**Objetivo:** Gerenciar usuários e estrutura de disciplinas.

**Dinâmica de Navegação:** Alternância entre abas de usuários e disciplinas.

**Requisitos Funcionais Atendidos:** RF11.

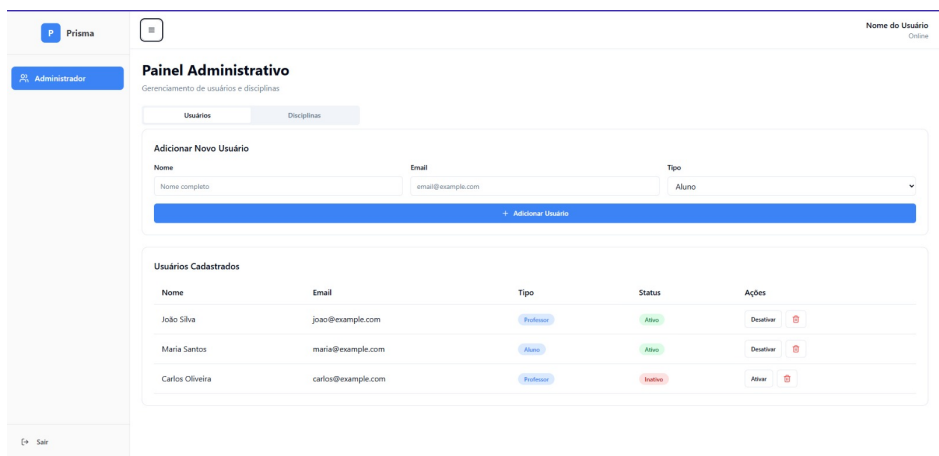


Figura 10: Painei administrativo

## **5 Análise do Sistema**

Este item deve apresentar a documentação da análise do sistema conforme o processo ou ciclo de vida descrito no capítulo 3. Organize o capítulo para apresentar os artefatos previstos e o que mais for necessário (protótipos, implementação, versões, telas, etc.), incluindo no mínimo (ajuste as subseções conforme necessário):

### **5.1 Modelo do Banco de Dados**

Modelo Conceitual/Lógico: Apresentar o esquema relacional gráfico (opcionalmente pode ser apresentado também o Diagrama Entidade-Relacionamento) do banco de dados normalizado, constando as tabelas com os atributos e restrições (chaves).

Dicionário de dados: Apresentar o dicionário de dados do banco de dados. Documentar cada tabela com seus atributos mostrando nome do atributo, tipo, tamanho, descrição, se é obrigatório ou não, e o que mais for necessário para descrever os dados. Documentar também usuários, stored procedures, funções e qualquer outra implementação ligada ao banco de dados.

### **5.2 Diagrama de Classes**

Apresentar o diagrama de classes previsto conforme a fase do projeto.

### **5.3 Diagrama de Atividades**

Apresentar o diagrama de atividades, que representa o detalhamento de tarefas e o fluxo de uma atividade para outra de um sistema. Nem todas as tarefas do sistema necessitam de um detalhamento, portanto deve-se considerar no que o diagrama irá auxiliar na implementação do sistema para decidir quais atividades devem ser descritas.

## **6 Implementação**

### **6.1 Descrição do código**

Descrever o sistema quanto ao código gerado. Explicar a organização dos arquivos, pacotes, classes ou quaisquer estruturas utilizadas no desenvolvimento do projeto, listando os componentes criados e sua estrutura. Use diagramas (Diagrama de Componentes, Diagrama de Pacotes) para ilustrar a implementação.

Descrever também convenções e padronizações para comentários no código, nomenclatura de classes, objetos, funções, etc. Se necessário, use exemplos.

### **6.2 Implantação**

Explicar passo-a-passo para execução do software desenvolvido. Citar os requisitos necessários, configuração e providências para execução do projeto entregue. Se necessário, usar links, referências ou indicação dos recursos imprescindíveis para execução.

### **6.3 Telas principais**

Apresentar as telas mais significativas do sistema, aquelas importantes para demonstração do seu funcionamento. Assim como nos protótipos, cada tela deve ser acompanhada de uma descrição sucinta de seu objetivo e sua dinâmica de navegação. O objetivo aqui é demonstrar o produto final.

## **7 Considerações Finais**

Apresentar e discutir os resultados obtidos e sua aplicabilidade. Abordar o que foi atingido e o que não foi, as limitações, possíveis integrações com outros projetos e continuação do sistema em trabalhos futuros.



## 8 Bibliografia

Listar todas as referências utilizadas no projeto.